UNIVERSIDAD D SEVILLA

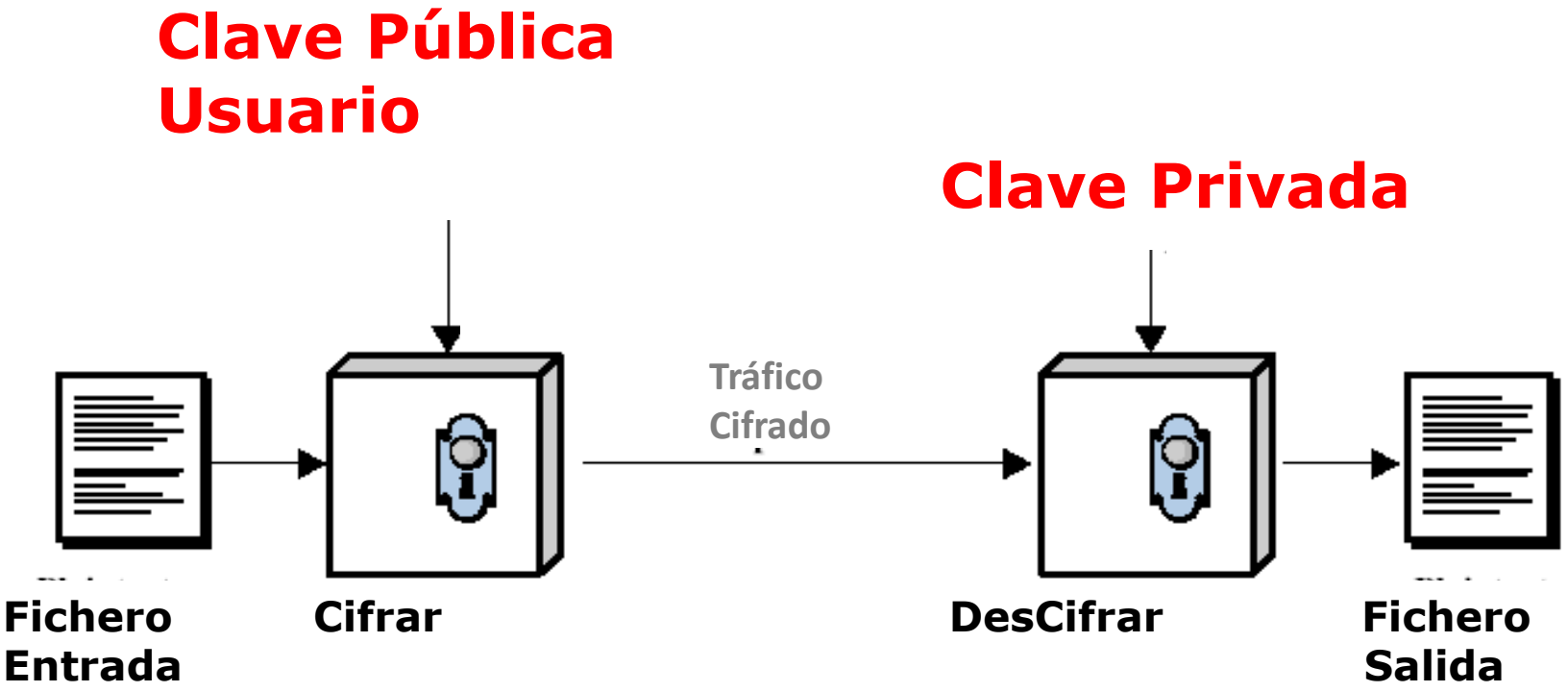
Tema 3. Confidencialidad de la Información (II)
Protocolos Seguros

Dr. Rafael Martínez Gasca
Grupo de Investigación **Quivir**,
Tecnologías Inteligentes y de Seguridad de los Sistemas de Información
Departamento de Lenguajes y Sistemas Informáticos
Universidad de Sevilla

Índice

- Conceptos Fundamentales
- Implementación de los requisitos de confidencialidad
  - Criptografía
    - Cifrado de clave simétrica
    - Cifrado de clave asimétrica
    - Protocolo de Acuerdo de Clave
    - Protocolo SSL/TLS, SSH, PGP y SRTP
    - Protocolo WEP, WAP y WAP2
    - XML Encryption
  - Esteganografía

Criptografía de clave pública



Implementación de la Confidencialidad.
Criptografía de clave pública

- **Public Key:** Asociada a una entidad particular (por ejemplo, un individio o una organización). Una clave pública se pretende que la conozca cualquiera que necesita realizar interacciones confiables con esa entidad.
- **Private Key: Supuestamene conocida solo por una entidad particular**. Siempre por tanto debe guardarse secreta. ***Está siempre asociada con la clave pública.***

Implementación de la Confidencialidad.
Criptografía de clave pública

- Java Criptography Extension(JCE)
- Clase ***KeyPairGenerator*** de java.security genera un objeto ***KeyPair***
- Clase ***KeyGenerator*** de *java.security* genera sólo un objeto ***SecretKey***
  - *El constructor esta protegido*
    - ***KeyGenerator kg= KeyGenerator.getInstance("DESede");***
    - ***kg.init(168); // tamaño de la clave para el algoritmo Triple Des***
    - ***SecretKey skey=kg.generateKey();***
    - ***skey.getFormat();*** aparece en forma de cadena
- Clase *KeyPairGenerator* se puede usar *getInstance(algoritmo);*
  - ***kpg.initialize(keySize);***
  - ***keys=kpg.genKeyPair();***
  - **Keys es un ojbeto del tipo *KeyPair***
  - ***keys.getPrivate();*** *//da un objeto* ***PrivateKey***
  - ***Keys.getPublic();*** *//da un objeto* ***PublicKey***

Implementación de la Confidencialidad.
Generación de claves

- Aleatoriamente generada usando un ***random number generator*** *(RNG)* o ***pseudorandom number generator*** *(PRNG)*.
  - El algoritmo ***PRNG*** produce datos que son aleatorios bajo análisis. ***PRNGs*** usa una ***seed*** para producir mejores resultados, esto hace que las condiciones iniciales del ***PRNG*** sean mucho más dificiles de adivinar por el atacante.
  - En Java la clase ***SecureRandom*** sirve para generar un número fuertemente pseudo random usando el Algoritmo PRNG
- Generada usando un ***passphrase*** y un ***key generation algorithm or function (KDF)***, generalmente usando una función de hashing seguro.

- RSA **(Rivest-Shamir-Adleman)** es el algoritmo muy usado (para cifrado y las firmas digitales).
- DSA (D**igital Signature Algorithm)**, es un estándar del Gobierno Federal de EEUU o FIPS para firmas digitales.
- **El Gamal** basado en el problema del logaritmo discreto.
- **Criptografía de Curvas Elípticas (pero no son elipses):** Una curva elíptica sobre ***R*** es el conjunto de puntos del plano (x,y) que cumplen: $y^2 = x^3 + ax + b$
  - *Elliptic curve cryptography*, ECC. Sus autores argumentan que puede ser más rápida y **usar claves más cortas que los métodos anteriores y menor hardware**, al tiempo que proporcionan un nivel de seguridad equivalente.

Implementación de la Confidencialidad.
Cifrado/Descifrado en Java

- Use an instance of javax.crypto.Cipher:

```
import java.security.*;
Cipher rsa = Cipher.getInstance("RSA/ECB/PKCS1Padding");
```

- Cifrado de texto

```
rsa.init(Cipher.ENCRYPT_MODE, publicKey);
byte[] ciphertext = rsa.doFinal("my cleartext".getBytes());
```

- Descifrado de texto

```
rsa.init(Cipher.DECRYPT_MODE, privateKey);
String cleartext = new String(rsa.doFinal(ciphertext));
```

1. **Las claves públicas deberían ser autenticadas:** No podemos estar seguros absolutamente que una clave pública pertenece a la persona que dice ser y así se deben verificar que los representan realmente.
2. **Es posible un compromiso de la seguridad generalizado:** Si un atacante determina la ***clave privada de una entidad (sobre todo una autoridad de certificación)***, sus mensajes podrán ser conocidos.

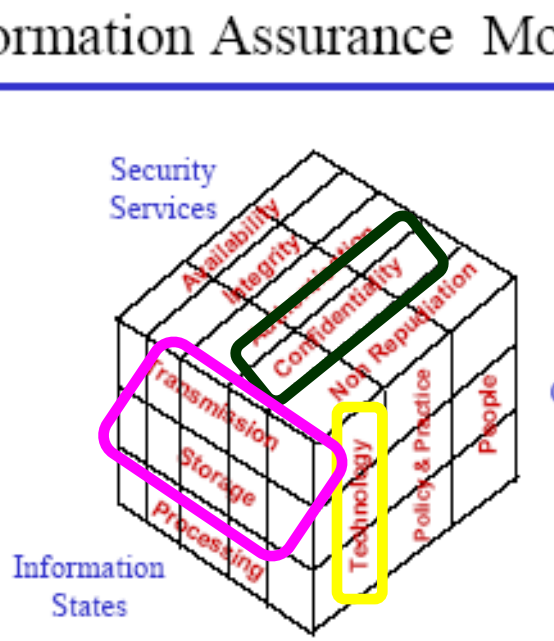
3. Pérdida de la clave privada: Esto significa que los mensajes recibidos no pueden descifrarse. (ataques de ransonware)

4. Lentitud: Más lento que el cifrado simétrico. No utilizable para descifrar mensajes masivos. (Son de 100 a 1000 veces menos eficientes que los de clave simétrica).

- Por ello se usa **intercambio seguro claves secretas para su uso en algoritmos de cifrado/descifrado.**

5. Las claves deben ser de mayor tamaño que las de la criptografía simétrica y el mensaje cifrado (criptograma) ocupa más espacio que el mensaje original.

- Introducción. Conceptos Fundamentales
- Requisitos de la seguridad de la información
- Implementación tecnológica de los requisitos



- Técnicas Criptográficas
  - Cifrado de clave simétrica
  - Cifrado de clave asimétrica
  - Protocolo de Acuerdo de Claves
  - Protocolo SSL/TLS,SSH y PGP
  - Protocolo WEP, WAP y WAP2
  - XML Encryption
- Esteganografía

Implementación de la Confidencialidad
Protocolo de Acuerdo de Claves

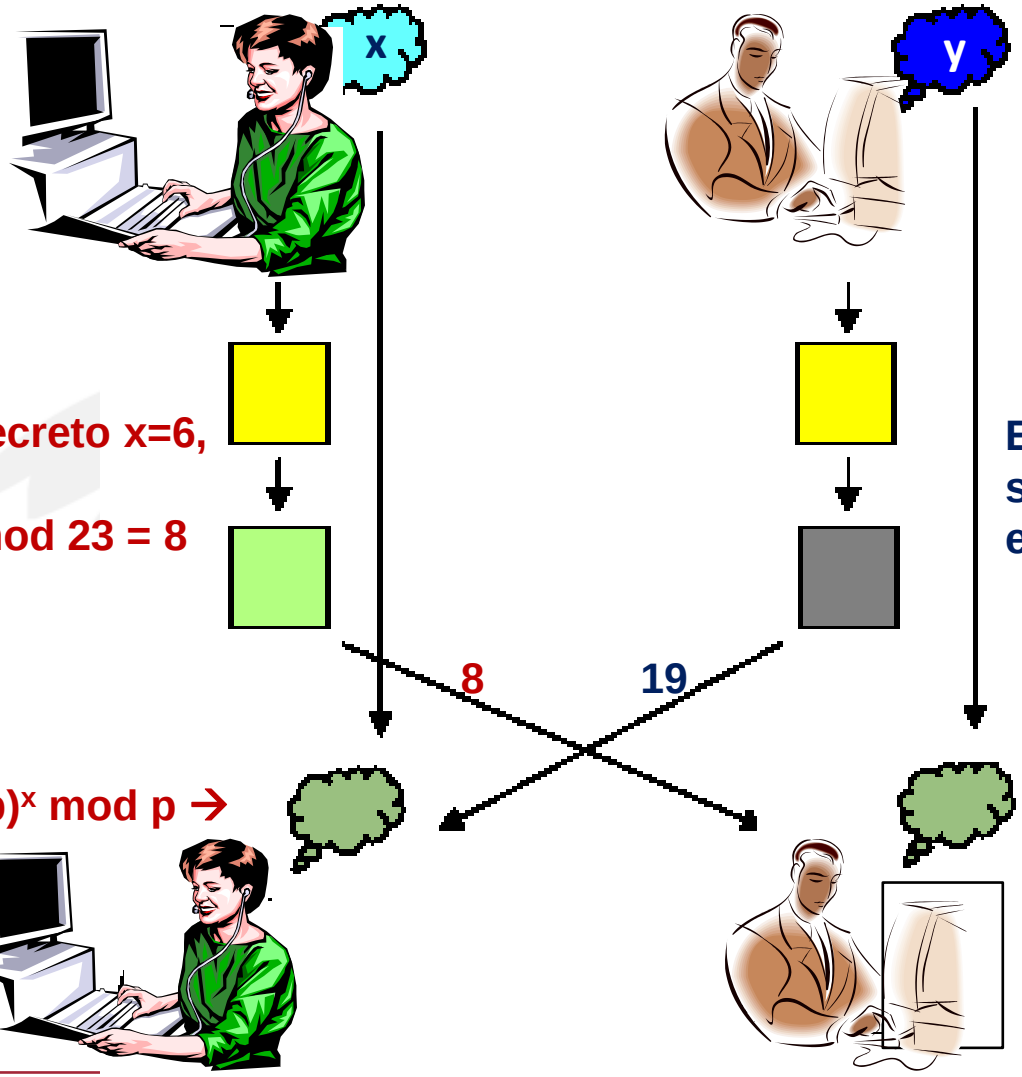
- Diffie-Hellman es un protocolo de establecimiento de claves entre partes que no han tenido contacto previo, utilizando un canal inseguro
- Se emplea generalmente como medio **para acordar claves simétricas que serán empleadas para el cifrado de una sesión** (establecer clave de sesión)
- Siendo no autenticado, sin embargo, es la base para varios protocolos autenticados
- Whitfield Diffie y Martin Hellman recibieron el premio Turing en el 2015

Implementación de la Confidencialidad
Protocolo de Acuerdo de Claves

- La idea es generar una clave compartida sin que un intruso pueda llegar a obtenerla
- Para ello se eligen **dos números públicos y un número secreto** para cada parte de la comunicación
- Cada parte hace una serie de operaciones con los dos números públicos y su número secreto, y **se intercambian los resultados** de forma pública
- Después cada uno por separado aplicarán una fórmula sobre la información compartida y el número secreto
- **Ambos llegarán al mismo número resultado**, que no habrá sido transmitido por el canal de comunicación

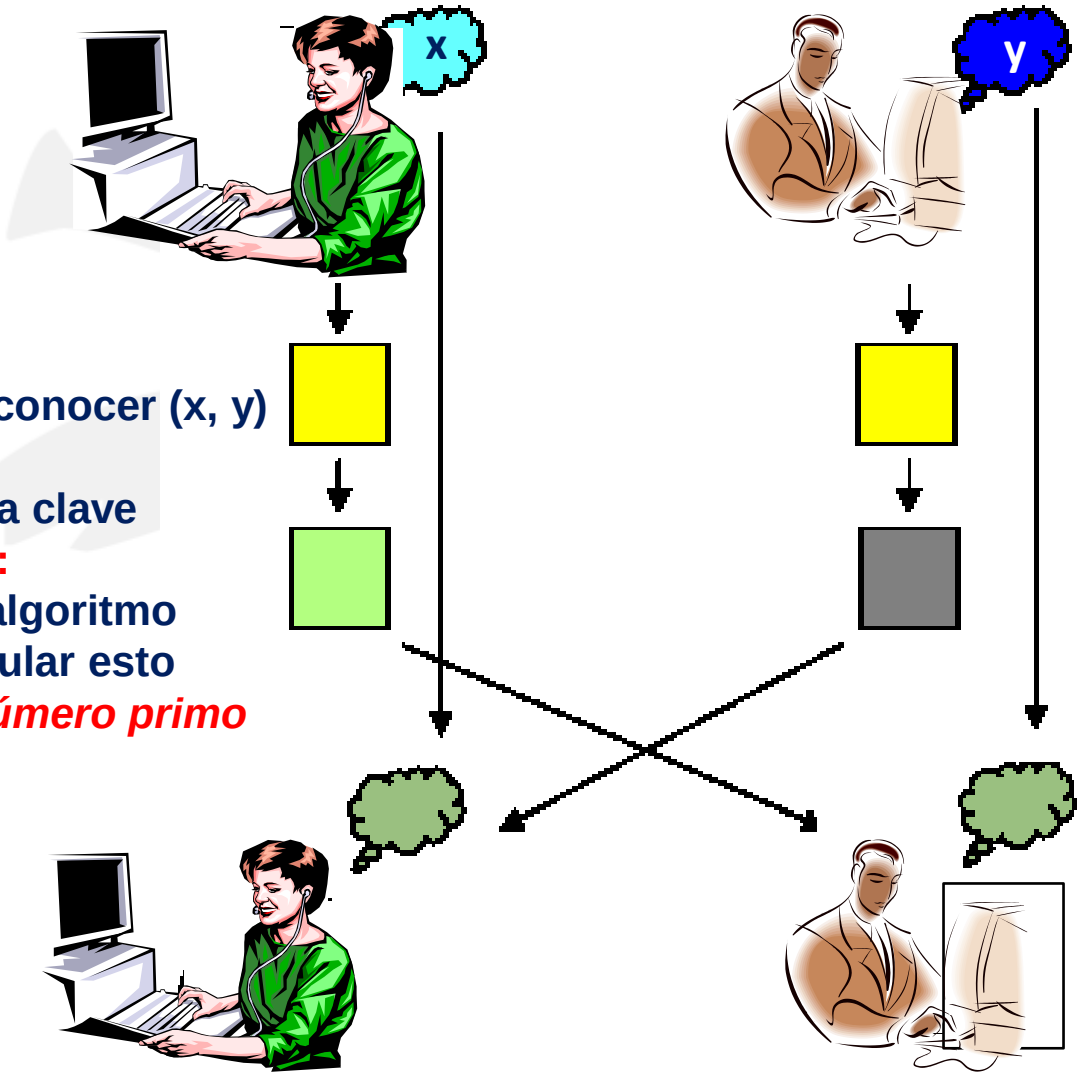
Implementación de la Confidencialidad.
Protocolo de Acuerdo de Claves

- **Ejemplo**: Inicialmente se acuerda usar el número primo p=23 y la base g=5



Clave obtenida: 2

Implementación de la Confidencialidad.
Protocolo de Acuerdo de Claves



Intrusos: Le falta conocer (x, y)

A y B comparten la clave secreta: $g^{xy} \bmod p$:
no se conoce un algoritmo práctico para calcular esto cuando *p es un número primo muy grande.*

- **Ataques**
  - Como p, g son conocidos, y también lo transmitido por el canal, se puede calcular uno de los dos **valores privados (x o y)** por fuerza bruta. Pero sería un problema intratable computacionalmente siempre que p sea un número primo grande de 200 o más dígitos
  - Otro posible ataque sería el de tipo **Man-in-the-middle.** Si la comunicación es interceptada por un tercero, éste se puede hacer pasar por el emisor cara al destinatario y viceversa, ya que no se dispone de ningún mecanismo para validar la identidad de los participantes en la comunicación

Implementación de la Confidencialidad.
Protocolo de Acuerdo de Claves

- Se ponen de acuerdo las partes para llevar a cabo la comunicación segura (Protocolo de acuerdo sobre claves de Diffie-Hellman. ***Anonymous Diffie-Hellman***)
- JCE soporta el acuerdo de claves con la clase ***KeyAgreement*** de ***javax.crypto***
  - ***KeyAgreement ka=KeyAgreement.getInstance("DH");***
- Cada una de las partes deberá crear e inicializar un objeto ***KeyAgreement***
- Posteriormente cada una de las partes implicadas en el acuerdo sobre claves ejecuta las diferentes fases del acuerdo, que varían dependiendo del protocolo.

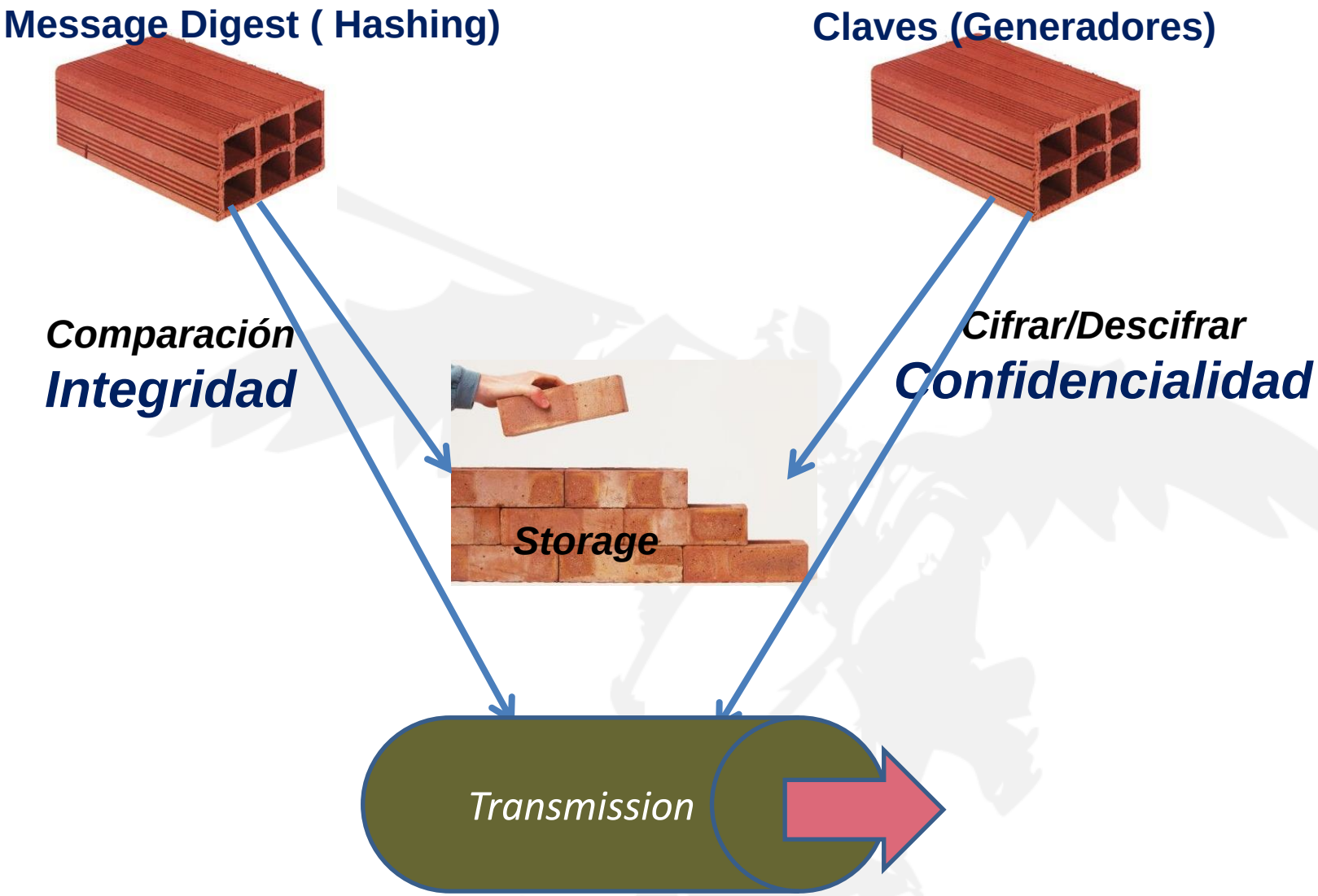
Implementación de la Confidencialidad.
Protocolo de Acuerdo de Claves

- SKIP (**Simple Key Management for Internet Protocols**)
  - Se necesita la base SKIP *g* y el módulo SKIP
  - Se emplea el siguiente módulo de 1024 bits
    - F488FD584E49DBCD20B49DE49107366B336C380D451D0F7C88B31C7
      C5B2D8EF6F3C923C043F0A55B188D8EBB558CB85D38D334FD7C175
      743A31D186CDE33212CB52AFF3CE1B1294018118D7C84A70A72D686
      C40319C807297ACA950CD9969FABD00A509B0246D3083D66A45D419
      F9C7CBD894B221926BAABA25EC355E92F78C7
  - Se suele usar para **soportar VPN (Redes Privadas Virtuales)**

Índice

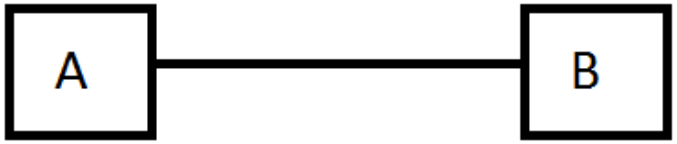
- Conceptos Fundamentales
- Implementación de los requisitos de confidencialidad
  - Criptografía
    - Cifrado de clave simétrica
    - Cifrado de clave asimétrica
    - Protocolo de Acuerdo de Clave
    - Protocolo SSL/TLS, SSH, PGP y SRTP
    - Protocolo WEP, WAP y WAP2
    - XML Encryption
  - Esteganografía

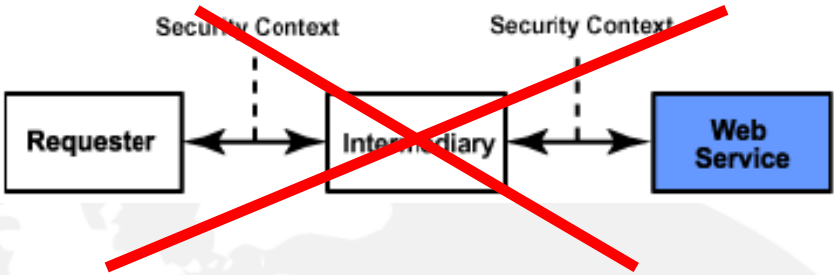
Implementación de la Integridad y Confidencialidad Technology



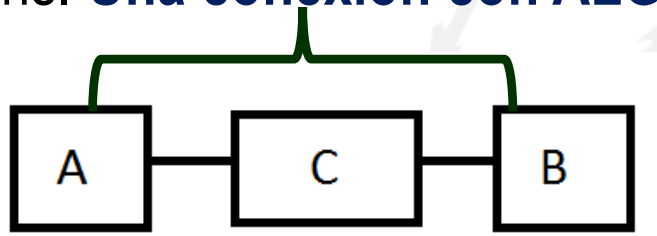
Implementación de la Confidencialidad
Transmisión

- Requisitos: Confidencialidad e integridad en las transmisiones
  - **Point to Point- Transport level security** basado en Secure Sockets Layer (SSL), **CONEXIÓN DIRECTA**





  - **End-to-End.** Message level security is an application layer service and facilitates the protection of message data between applications. **Una conexión con ALGO POR EL MEDIO.**

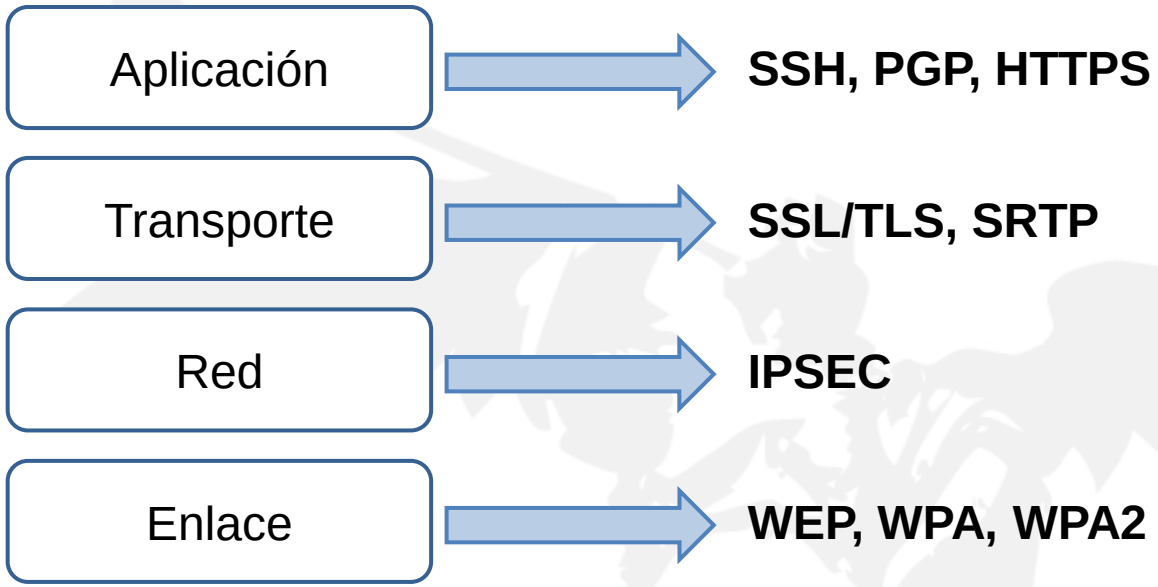


Implementación de la Confidencialidad.
Transmisión

- **Transport-Level Security**
  - ***SSL/TLS*** permite que dos aplicaciones se comuniquen de forma segura a través de las redes y se autentiquen entre ellas.
  - Usado para proporcionar **point-to-point security, integridad, confidencialidad y autenticación.**

- **Message-Level Security**
  - Para un WS, ***XML Encryption*** proporciona la seguridad deseada.
  - Gestiona la seguridad a nivel de aplícación lo que significa que ***la seguridad a otros niveles es innecesaria.***

Implementación de la Confidencialidad.
Transmisión

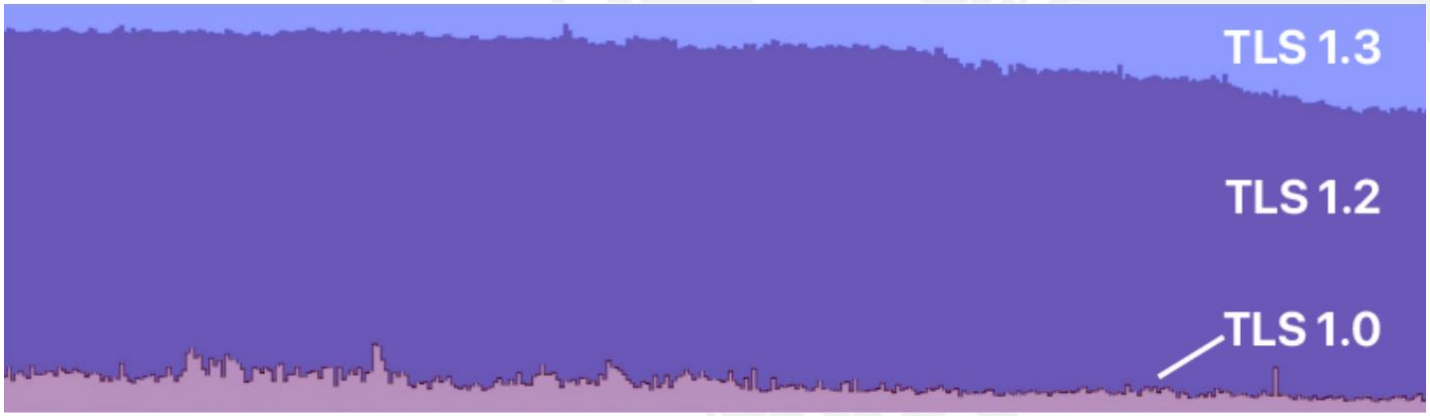


Implementación de la Confidencialidad.
Transmisión. SSL/TLS Protocol

- Arquitectura cliente/servidor para las transacciones seguras
- Secure Sockets Layer (SSL)
  - Netscape desde 1995
  - Proporciona Confidencialidad (Key size: 128 bit), Integridad de datos (MAC) y A**utenticación de servidor o mutua con certificados X.509**
  - Último SSLv3 (ataque POODLE en Octubre de 2014)
  - TLS: Transport Layer Security
  - IETF (Internet Engineering Task Force): Consorcio de colaboración técnica más importante en Internet
  - Basado en SSLv3
  - Últimas versiones
    - TLS 1.1 (RFC 4346 & RFC 4366)
    - TLS 1.2 (definido en RFC 5246) Agosto del 2008. Usa ya SHA-256 y ha sido actualizada por: **RFC**5746, RFC5878, RFC6176

- Porcentaje de conexiones que negocian TLS 1.3 a través de implementaciones de browser a fecha de Agosto 2019.

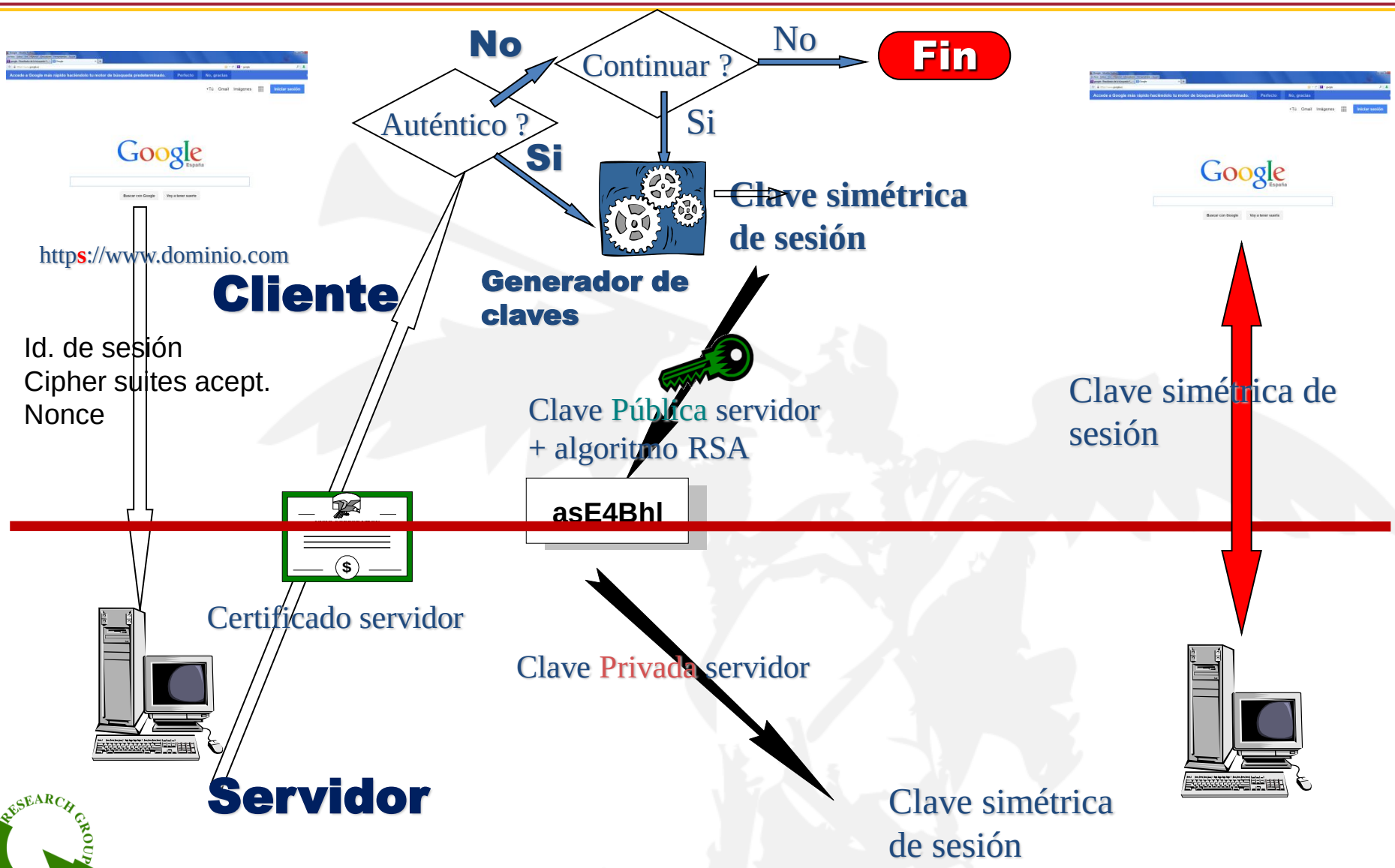
Browser	Porcentaje de TLS 1.3
Chrome	30%
Firefox	27%
Safari	27%



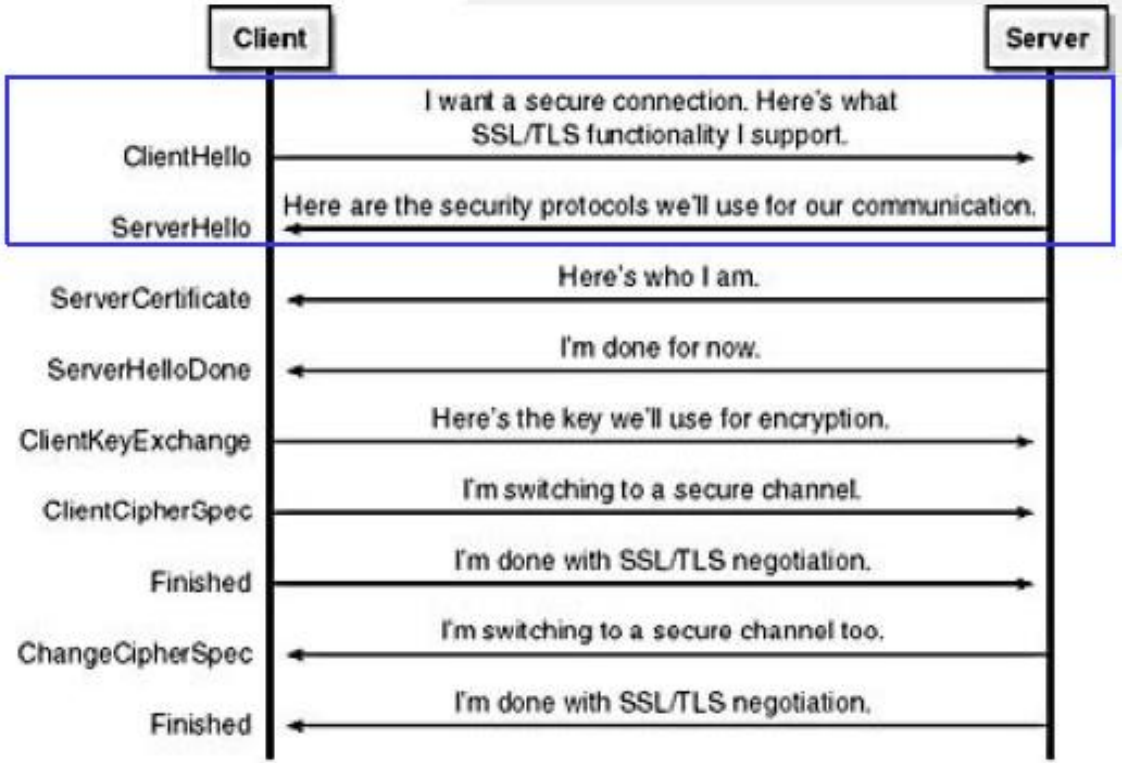
TLS version trends as seen by Cloudflare from May 2018 to May 2019

- **Para su diseño**
  - Uso del ***Principio de economía de mecanismos***
    - Los mecanismos de seguridad deben ser tan simple como sea posible (ya visto los algoritmos de clave pública son más lentos que los de clave simétrica, SSL/TLS y SSH usan una combinación de ambos).
    - Una parte recibe la clave pública del otro, y cifra una pequeña cantidad de datos (bien una clave simétrica o alguno dato usado para generarla). El resto de la conversación usa un algoritmo de clave simétrica
  - Uso del ***Principio de aceptación psicológica***
    - Los mecanismos de seguridad no deberán hacer más difícil el acceso a los recursos

Implementación de la Confidencialidad.
Transmisión. SSL/TLS Protocol



Implementación de la Confidencialidad.
Transmisión. SSL/TLS Protocol



Comprobación SSL: http://www.sslshopper.com/ssl-checker.html
Decodificador de certificados: http://www.sslshopper.com/csr-decoder.html

- Java Secure Socket Extension(JSSE)
  - Los algoritmos de cifrado, acuerdo de claves y autenticación incluyen 3DES, Diffie-Hellman y DSA.
- Sockets de servidor SSL
  - Objetos ***SSLServerSocket*** *que escuchan conexiones SSL en el puerto 7070.*
  - Usa el patrón de diseño Factoría con la clase ***SSLServerSocketFactory*** para construir ***SSLServerSocket***
    - **Llamar al método *createServerSocket***
  - Se deberá diseñar un método que escuchará las peticiones de los clientes y comprobará su login y password.
  - El uso de la clase ***SSLServerSocket*** es idéntico al uso de la ***ServerSocket***

```
import java.io.*;
import javax.net.ssl.*;

public class LoginServer {
    private static final String CORRECT_USER_NAME = "Rafael";
   private static final String CORRECT_PASSWORD = "H89rtaW_16K";
     private SSLServerSocket serverSocket;
    // Constructor
     public LoginServer() throws Exception
     {
      // SSLServerSocketFactory para construir los SSLServerSockets
      SSLServerSocketFactory socketFactory = ( SSLServerSocketFactory )
           SSLServerSocketFactory.getDefault();
       // creación de un objeto SSLServerSocket en el puerto 7070
      serverSocket = ( SSLServerSocket )  socketFactory.createServerSocket( 7070 );
     }
```

```
// ejecución del servidor para escuchar peticiones de los clientes
  private void runServer()
   {
       while ( true ) {
             // espera las peticiones del cliente para comprobar username/password
          try {
               System.err.println( "Esperando conexiones de clientes..." );
              SSLSocket socket =  ( SSLSocket ) serverSocket.accept();
               // abre un BufferedReader para leer los datos del cliente
             BufferedReader input = new BufferedReader(new InputStreamReader(
socket.getInputStream() ) );
                  // abre un PrintWriter para enviar datos al cliente
             PrintWriter output = new PrintWriter(new
OutputStreamWriter(socket.getOutputStream() ) );
```

```
// se lee del cliente el username y el password
        String userName = input.readLine(); String password = input.readLine();
        if ( userName.equals( CORRECT_USER_NAME ) &&  password.equals(
CORRECT_PASSWORD ) ) {
            output.println( "Bienvenido, " + userName );
        }
        else {
            output.println( "Login Incorrecto." );
        }
        output.close();
        input.close();
        socket.close();
        }
        catch ( IOException ioException ) {   ioException.printStackTrace(); }
        }
}
```

```
// ejecucion del servidor
  public static void main( String args[] ) throws Exception
  {
    LoginServer server = new LoginServer();
    server.runServer();
  }
}
```

Implementación de la Confidencialidad.
Transmisión. SSL/TLS en Java

- Sockets de clientes SSL
  - Usa la clase ***SSLSocket*** para comunicarse con el Servidor
  - Usa el patrón de diseño Factoría con la clase ***SSLSocketFactory*** para construir ***SSLSocket***
  - Se crea un ***PrintWriter*** para enviar el login y password al servidor
- Habilitar SSL requiere que ***LoginServer*** use un certificado que ***LoginClient*** confie.
- Usar keytool para generar un nuevo certificado
  - ***Keytool –genkey –keystore SSLStore –alias SSLCertificate***

```
import java.io.*;
import javax.swing.*;
import javax.net.ssl.*;

public class LoginClient {
    public LoginClient()
  {
    // Constructor que abre una conexión SSLSocket para enviar login/password al servidor
     try {
          SSLSocketFactory socketFactory =  ( SSLSocketFactory ) SSLSocketFactory.getDefault();
          SSLSocket socket =  ( SSLSocket ) socketFactory.createSocket("localhost", 7070 );
            // crea un PrintWriter para enviar login/password al servidor
        PrintWriter output = new PrintWriter(new OutputStreamWriter( socket.getOutputStream() ) );
        String userName = JOptionPane.showInputDialog(null,"Introduzca su UserName:" );
        output.println( userName );  // envio del username al servidor
        String password = JOptionPane.showInputDialog( null,"Introduzca el Password:" );
        output.println( password );
        output.flush();
```

```
// crea un objeto BufferedReader para leer la respuesta del servidor
    BufferedReader input = new BufferedReader(new InputStreamReader( socket.getInputStream ()) );
    String respuesta = input.readLine(); // lee la respuesta del servidor
   JOptionPane.showMessageDialog( null, respuesta); // muestra la respuesta al cliente
    output.close();
      input.close();
      socket.close();
   } // end try
    catch ( IOException ioException ) {
            ioException.printStackTrace();
    }
     // Salida de la aplicacion
    finally {
      System.exit( 0 );
    }
  }
```

```
// ejecución del cliente de acceso
  public static void main( String args[] )
  {     new LoginClient();
  }
}
```

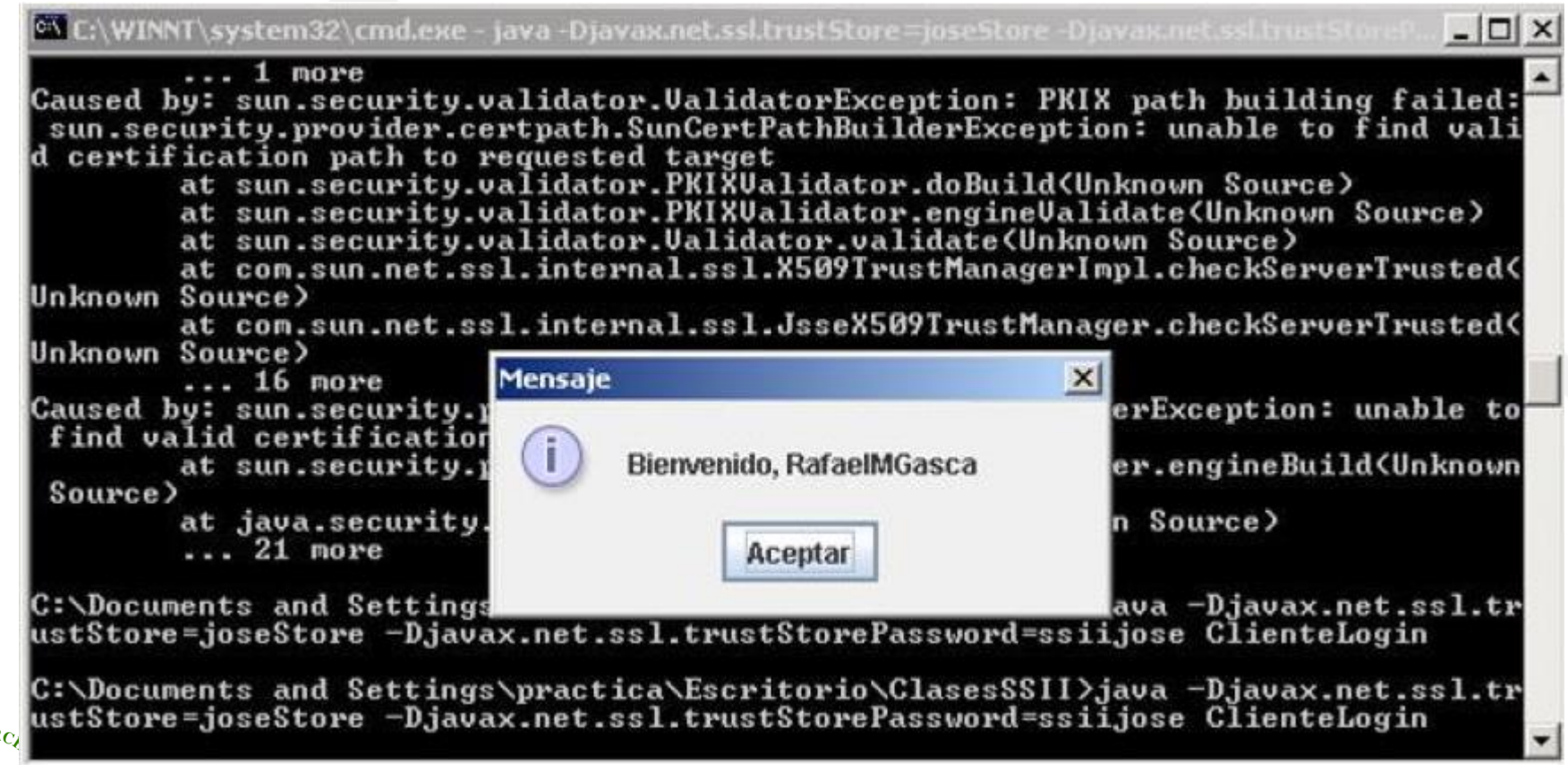
- SSL requiere que *LoginServer* use un certificado en el que el cliente *LoginClient* confia.
  - Usar keytool para generar el nuevo certificado y el keystore
    - ***Keytool –genkey –keystore SSLAlmacen –alias SSLCertificado***
- Ejecutar *LoginServer* especificando el keystore que contiene el certificado.



- Finalmente ejecuta el *LoginClient* y especifica el *truststore* para ese cliente, para validación del certificado



- Resultado de un login/password correcto.



- OpenSSL
  - Proyecto colaborativo para desarrollar una robusta, commercial-grade, full-featured, y Open Source toolkit para implementar los protocolos Secure Sockets Layer (SSL v2/v3) y Transport Layer Security (TLS v1) , así como una **biblioteca criptográfica de propósito general.**
  - OpenSSL se basa en la biblioteca SSLeay.
  - The OpenSSL toolkit es libre para usar en fines comerciales y no-comerciales sujeto a algunas restricciones simples.
  - La versión ***OpenSSL 3.0*** está disponible para testearla.
  - **https://www.openssl.org/news/vulnerabilities.html**